Monte Carlo Areas and Integrals: Exercise Guide for 3A

Maryam Ahsan

Mathematical Methods for Physicists II

Python Notebook Guide

Contents

1 How to use this exercise guide 1

2 Exercise 1: estimate pi from circle hits 1

3 Exercise 2: vary the number of points 1

4 Exercise 3: repeat a fixed-size experiment 2

5 Exercise 4: estimate an ellipse in two ways 2

6 Exercise 5: estimate oscillatory integrals 3

7 Exercise 6: measure typical error rather than one error 4

8 Checks for an independent implementation 4

This is a collection of notes based on my Mathematical Methods for Physicists II coursework on Monte Carlo area estimates and integration. It accompanies notebook 3A and explains each exercise's mathematical target, computational logic, and output interpretation. The shared Monte Carlo foundations guide should be read first.

1 How to use this exercise guide

Read *Monte Carlo Methods: Concepts and Computational Foundations* first. This guide applies its indicator, sample-average, uncertainty, and RMS-error ideas to each section of notebook 3A. It preserves the coursework's original experiments, including its fine-grid area sampler and irregular sample-size choices.

A useful habit is to write the sampling region, membership test or integrand, estimator, and expected uncertainty before coding. Then identify which output can check each ingredient. The known values of π and 2π are benchmarks here; the algorithms estimate them through hits rather than substituting the answers.

2 Exercise 1: estimate pi from circle hits

2.1 Geometry and the estimator

The square $[-1, 1) \times [-1, 1)$ has area four, and the unit disk has area π. For a uniformly drawn point the hit probability is $p = \pi/4$ in the continuous model. Define a hit by $x^2 + y^2 \leq 1$ and estimate π as $4n_{\text{hit}}/n$.

The notebook draws integers from -1000 through 999, then divides them by 1000. Each coordinate therefore lies on a grid with spacing 0.001. Its hit probability is a discrete approximation to $\pi/4$. The endpoint asymmetry follows from the upper-exclusive integer generator; the key distinction is discrete versus continuous sampling, not whether a continuous box includes its edge.

2.2 Translate the calculation into an array algorithm

1. Draw separate arrays of n random x and y coordinates.
2. Compute $x^2 + y^2$ element by element.
3. Apply the circle test to obtain a Boolean array.
4. Sum the Boolean values to count hits.
5. Divide by n and multiply by four.

A Boolean True contributes one and False contributes zero. The coordinate pairs, not two unrelated histograms, are the sampled points. Return the estimate from a reusable function.

For a visual check, use a separate sample of 1000 points and distinguish inside and outside points. The printed estimate for that picture must use *those* hits, as the notebook does. The picture confirms classification but does not establish statistical accuracy.

2.3 Checks and interpretation

Every coordinate must be in the declared interval, every hit must satisfy the test, and the estimate must lie between zero and four. Equal x and y scaling is needed for a geometrically faithful circle. Boundary inclusion affects finitely many discrete grid points; for continuous random coordinates the boundary has probability zero.

The exact expectation of the current grid estimator is 3.141547 rather than precisely π. Its tiny bias is retained and explained rather than changing the original sampler. Repeating more points at the same spacing cannot remove this systematic offset.

3 Exercise 2: vary the number of points

Two lists supply trial sizes. The first includes 100 through 550,000 points and deliberately repeats 100,000; the second uses irregular sizes from the original homework. Each call draws a fresh batch, rather than extending the preceding sample.

Construct an estimate list by looping over the sizes and calling the circle function. Plot estimate against size and include the reference π. The logarithmic horizontal scale in the first plot makes very different sample

sizes visible together; it does not transform the estimator itself.

The typical scatter should become smaller at larger n. Individual points need not move monotonically toward the line. The duplicated size is another random estimate, not an error in the list. For the largest listed size, 804,600, the continuous-model one-run standard deviation is about 0.00183, much larger than the grid offset of about 4.57×10^{-5}.

A single trial at each size cannot accurately establish a scaling exponent. The repeated RMS experiment later supplies a more informative comparison. Preserve the distinction between one realization and an expected statistical trend.

4 Exercise 3: repeat a fixed-size experiment

4.1 The two levels of sample count

Perform $R = 50$ separate estimates, each with $n = 10000$ points. Store one scalar estimate per run; the resulting list has 50 entries, not 10,000. Initialize the random stream once, without resetting it inside the loop.

Calculate the run mean, sample variance with `ddof=1`, standard deviation, and standard error of the run mean:

$$\overline{\pi} = \frac{1}{50}\sum_r \widehat{\pi}_r, \quad s_\pi^2 = \frac{1}{49}\sum_r (\widehat{\pi}_r - \overline{\pi})^2, \quad s_\pi = \sqrt{s_\pi^2}, \quad \mathrm{SE}_{\mathrm{mean}} = s_\pi/\sqrt{50}.$$

The continuous-theory single-run spread is about 0.0164, and the spread of the 50-run mean is about 0.00232. The observed estimates will not equal these approximate values exactly.

4.2 Read the histogram correctly

The histogram displays the distribution of *individual run estimates*. Its width corresponds to s_π, not to $s_\pi/\sqrt{50}$. Twelve bins and 50 runs give a noisy shape; an uneven histogram is not surprising.

Compare its general centre with the reference line, but do not expect the line to pass through the tallest bin. A roughly normal distribution is motivated by large binomial hit counts. The mean's smaller standard error measures sampling uncertainty and excludes the fixed-grid bias.

5 Exercise 4: estimate an ellipse in two ways

5.1 Define the direct problem

The ellipse is $x^2/4 + y^2 \le 1$, with semiaxes $a = 2, b = 1$. Its bounding rectangle has width four, height two, and area eight. The test used in code,

$$x^2 + 4y^2 \le 4,$$

is algebraically equivalent. The direct estimate is $8n_{\mathrm{hit}}/n$.

Draw x-grid values by dividing integers from -2000 through 1999 by 1000 and y-grid values using -1000 through 999. Apply the test and multiply the hit fraction by eight.

5.2 Define the rescaled-circle problem

The analytic area relation is $A = ab\pi = 2\pi$. Estimate π from the unit-circle experiment and multiply that *random estimate* by two. This second route is not an exact answer because its input is estimated, not the constant `np.pi`.

For continuous sampling, stretching a unit-disk box by $x \mapsto 2x$ maps it into the ellipse box. The two hit probabilities are then identical, so at the same n the area estimators have the same distribution and variance. Multiplying an estimator by two multiplies its variance by four and standard deviation by two.

The current discrete grids are slightly different: after dividing direct x coordinates by two, their spacing is 0.0005, whereas the circle x grid has spacing 0.001. Thus exact discrete expectations need not agree perfectly;

both approximate the same continuous geometry.

5.3 Run, summarize, and interpret

Run 50 independent estimates of 50,000 points for each method. Store the two series separately, using separate random batches. For each calculate mean, sample variance, sample standard deviation, and the mean's standard error. Overlay their histograms with the benchmark 2π.

Different displayed means or variances can reflect finite-run fluctuations, not intrinsic superiority of one method. The histograms need not coincide point by point. Averaging over more runs narrows the uncertainty of each mean but does not remove its grid bias.

6 Exercise 5: estimate oscillatory integrals

6.1 Construct the function-value estimator

Unlike the area sampler, the integration function uses continuous uniform coordinates on the interval. For $X \sim \text{Uniform}(a, b)$, the target is $(b-a)\mathbb{E}[f(X)]$. Therefore:

1. Draw n independent coordinates on $[a, b]$.
2. Evaluate the requested function at each coordinate.
3. Take the sample mean of those values and multiply by interval width.
4. Estimate standard error as width times sample standard deviation divided by $\sqrt{n}$.

At least two values are needed for the stated sample-standard-deviation estimate. The notebook's chosen sample sizes satisfy this. A function selector distinguishes the two integrands and raises an error for an unknown choice.

6.2 The signed sine and its benchmark

For $f(x) = \sin(100x)$ over $[0, 2\pi]$,

$$I = \left[-\frac{\cos(100x)}{100}\right]_0^{2\pi} = 0.$$

The interval contains 100 complete periods. With 100,000 random coordinates, the printed result fluctuates around zero with an estimated standard error near the theoretical 0.0140. A nonzero estimate is compatible with a zero exact integral. A percentage error relative to zero would be undefined; compare absolute error with sampling uncertainty instead.

The integrand takes positive and negative values. Its mean estimates a signed integral, not the positive geometric area under its absolute value.

6.3 The second integrand and the notebook's output scope

The same function also computes $|\sin(100x)|^x$ with 1000 points. The absolute value keeps its base nonnegative for the noninteger exponent. The function lies between zero and one on this nonnegative interval, so its integral lies between zero and 2π. It does not share the sine integral's exact zero value.

At the exact endpoint $x = 0$, the expression involves 0^0; any choice at this single point does not affect the continuous integral. NumPy follows its numerical power convention. This is separate from the appreciable cancellation fluctuations in the signed sine.

The present notebook stores the second estimate and its standard error but prints only the signed-sine result. Do not interpret the displayed zero benchmark as a check on the second stored calculation. If you later display the second result, it needs its own reference or repeated-run assessment.

7 Exercise 6: measure typical error rather than one error

Use sizes $100, 500, 1000, 5000, 10000, 50000$. At each size draw 30 independent pi estimates. Calculate each squared error relative to π, then take their mean and its square root:

$$\widehat{\mathrm{RMS}}(n) = \sqrt{\frac{1}{30}\sum_{r=1}^{30}(\hat{\pi}_r - \pi)^2}.$$

This is a finite-run estimate of total RMS error, not the standard error of the mean of those 30 estimates.

Calculate the continuous-theory sampling spread

$$4\sqrt{\frac{p(1-p)}{n}}, \qquad p = \pi/4,$$

and compare both curves on logarithmic axes. The predicted slope is $-1/2$; doubling the sample count decreases sampling spread by $1/\sqrt{2}$, not by one half.

The 30-run RMS points can fluctuate around the reference curve. The dashed curve omits discrete-grid bias, so equality is approximate rather than exact. At much larger sample counts the grid-related RMS floor would matter. No Markov chain or correlated thermal configuration enters this independent-sampling experiment.

8 Checks for an independent implementation

Calculation	Essential check	Interpretation trap
Circle	Uniform box and $x^2 + y^2 \leq 1$	Larger single trial need not be better
Repeated pi	Independent batches and $R - 1$ variance	Histogram width is not mean's SE
Ellipse	Correct rectangle area and rescaled test	Different random histograms do not rank methods
Integral	Correct signed function and interval width	Zero benchmark excludes relative-percentage error
RMS scaling	Repeat each size and average squared errors	Theory is sampling spread, not a bias-inclusive bound

The physical application of these tools in 3B adds interactions and nonuniform thermal weights. Read the foundations' Markov-chain and correlation sections before using its independent-sample uncertainty formulas on spin measurements.